

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) provides a strong quality–efficiency trade-off, but its rendering pipeline depends on a local approximation that represents each 3D Gaussian by a 2D projected Gaussian. This approximation induces systematic geometric error, particularly for generic cameras with large field-of-view (FoV) mappings. Although several extensions have addressed non-pinhole imaging models, existing methods do not jointly provide *projective exactness* and *real-time efficiency* for general cameras. We present 3DGEER, a Gaussian rendering framework whose rendering and association components are specified directly in projective geometry. Starting from the continuous 3D Gaussian density, we derive a closed-form ray integral that supports exact forward evaluation and differentiable optimization for arbitrary camera models. For efficient execution, we introduce the *Particle Bounding Frustum (PBF)* for tight ray–Gaussian association without BVH traversal, together with the *Bipolar Equiangular Projection (BEAP)* for unified FoV parameterization, accelerated association, and improved supervision. Experiments on pinhole and fisheye datasets demonstrate that 3DGEER improves all reported metrics over prior methods, executes $5\times$ faster than existing projective exact ray-based baselines, and extrapolates to wider FoVs than those observed during training, yielding a new state of the art for real-time radiance field rendering.

1 INTRODUCTION

Volumetric particle rendering methods based on splatting, beginning with 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), have become widely adopted because they combine high visual fidelity with high rendering throughput (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024). Their efficiency is primarily obtained from EWA splatting (Zwicker et al., 2001), which replaces the true nonlinear projection of a 3D Gaussian with a locally affine 2D Gaussian on the image plane. This approximation is adequate for many narrow field-of-view (FoV) settings, but it produces substantial geometric error for wide-FoV cameras, where nonlinear projection and distortion cause the image-domain footprint to depart from a symmetric 2D Gaussian. The failure mode is especially relevant for fisheye imagery in robotics and autonomous-driving systems (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019, 2024; Sekkat et al., 2022). Recent differentiable-rendering extensions support either specific fisheye models (Liao et al., 2024) or broader camera families (Huang et al., 2024b). However, these methods still instantiate rendering through 2D projective-space approximations, and therefore do not provide an exact closed-form treatment of the projected 3D Gaussian density.

A complementary strategy is to reduce splatting error by increasing the number of primitives. For example, Celarek et al. (2025) evaluates whether exact rendering is necessary by scaling the Gaussian count up to 1M. Those results, however, are obtained on small object-centric scenes. In large-scale reconstruction, the number of primitives required to approximate geometry and radiance varies strongly across objects and view frustums, so uniform scaling becomes computationally expensive. Moreover, our experiments show that generic distorted cameras induce larger projective errors, and that increasing the Gaussian count alone does not eliminate the observed quality gap (see Fig. 5).

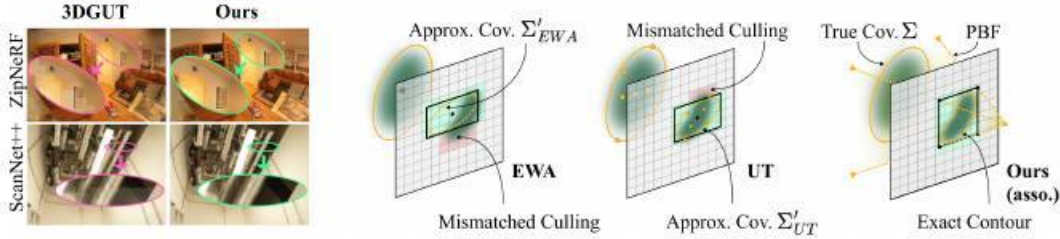


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts induced by inaccurate UT association. (Right) Schematic comparison of the proposed association procedure with alternative approximations. Our method computes the bounding structure directly from the true 3D covariance and therefore avoids an intermediate conic approximation.

Another line of work formulates volumetric particle rendering directly as ray tracing or ray marching (Yu et al. 2024b; Condor et al. 2025; Mai et al. 2025; Moënné-Loccoz et al. 2024; Talegaonkar et al. 2025; Gu et al. 2025). Such formulations are naturally compatible with large-FoV camera models and avoid projection-induced approximation during rendering. Their main bottleneck is computational: each camera ray must be associated with the subset of 3D particles that can contribute non-negligible density. Prior systems accelerate this association with advanced Bounding Volume Hierarchies (BVH), as in EVER (Mai et al. 2025) and 3DGRT (Moënné-Loccoz et al. 2024). Nevertheless, the traversal complexity and limited parallel regularity of pure ray-based pipelines prevent them from matching the throughput of 3DGS. 3DGUT (Wu et al. 2025) instead combines ray-based rendering with rasterization-based association. This hybrid design improves association efficiency, but the association stage inherits projective approximation error and can therefore violate projective exactness, producing grid-line artifacts in wide-FoV settings (see Fig. 1 Left). Consequently, existing methods have not simultaneously achieved the geometric exactness of ray-based rendering and the execution efficiency associated with splatting.

We therefore propose 3DGEER, a 3D Gaussian Exact and Efficient Rendering method that enforces *exactness in projective geometry* (defined in Sec. A) while retaining *high efficiency comparable to splatting* for generic cameras (both perspective and a variety of fisheye projection models). The method consists of three technical components: (i) We rederive the heuristic maximum response function (Keselman & Hebert 2022) for Gaussians from first principles and identify its relationship to projective exactness. This derivation yields a numerically stable differentiable rendering framework. (ii) For efficient ray-particle association under exact geometry, we reformulate the mapping as an association between Camera Sub-frustums (CSF) and Particle Bounding Frustums (PBF), which is structurally analogous to the tile-AABB mapping in 3DGS. For each 3D Gaussian, we derive a closed-form PBF that tightly bounds its contributing rays and permits efficient parallel association. (iii) We further define a Bipolar Equiangular Projection (BEAP) space in which supervision rays are sampled uniformly. This sampling parameterization aligns image-space partitions with the corresponding CSFs and empirically improves reconstruction quality relative to conventional pinhole or equidistant projections.

Experiments on fisheye and pinhole datasets, including ScanNet++ (Yeshwanth et al. 2023), ZipNeRF (Barron et al. 2023), Aria (Lv et al. 2025), and MipNeRF (Barron et al. 2021), show that 3DGEER consistently outperforms prior and concurrent methods. The gains are 0.9–4.8 PSNR under challenging wide-FoV evaluation and 0.3–2.0 PSNR on pinhole views. Notably, when trained only on narrow-FoV data, the proposed ray-based formulation still exceeds splatting-based methods on wide-FoV cameras, indicating stronger generalization to extreme-FoV rendering.

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

Most 3DGS variants (Yu et al. 2024a; Zhang et al. 2024; Rota Bulò et al. 2024; Gao et al. 2025; Zhao et al. 2025; Kerbl et al. 2024; Hou et al. 2025; Liu et al. 2025) retain the original EWA splatting procedure (Zwicker et al. 2001; Ren et al. 2002): each 3D Gaussian is projected to the image plane and represented by an approximate 2D conic. To extend differentiable rendering to distorted large-FoV cameras, FisheyeGS (Liao et al. 2024) and GS++ (Huang et al. 2024b) compute Jaco-

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The upper block contains splatting-based methods, whereas the lower block reports ray-based methods. Several ray-based methods still use projectively approximated association, such as EWA or the Unscented Transform.

Method	Render	Particle Asso. Method	Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS [Kerbl et al., 2023]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS [Liao et al., 2024]	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ [Huang et al., 2024b]	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS [Huang et al., 2024a]	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF [Yu et al., 2024b]	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER [Mai et al., 2025]	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT [Moënné-Loccoz et al., 2024]	Ray	BVH	Scene	3D Gauss.	✓	✗	✓
3DGUT [Wu et al., 2025]	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS [Hahlbohm et al., 2025]	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

biases of equidistant or spherical projection functions and use them to update the projected 2D mean and covariance. These methods nevertheless remain first-order approximations of a strongly non-linear mapping. In wide-FoV regimes, neglected higher-order terms become visible in the footprint geometry and consequently reduce reconstruction quality.

Formally, the exact projection of a 3D Gaussian through a nonlinear camera model, including both ideal pinhole and distorted projections, is not generally a symmetric 2D Gaussian. Therefore, any framework that substitutes the true transformation with a linear projective approximation introduces residual geometric error that cannot be removed within that approximation class. Our method instead evaluates Gaussian density along camera rays in a volumetric rendering formulation, thereby avoiding the 2D projection approximation in the rendering computation.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) assumes that a ray interacts with a Gaussian density through the point of maximum intensity response. Using this assumption, 3DGRT [Moënné-Loccoz et al., 2024] constructs a 3D Gaussian ray-tracing framework that samples in ray space without explicit image-plane projection, thereby reducing wide-FoV distortion artifacts. The resulting system, however, incurs substantial computational cost because ray-particle association depends on scene-level BVH spatial partitioning, which leads to low frame rates.

EVER [Mai et al., 2025] takes a different route by replacing Gaussian density with constant-density ellipsoids, which admits a closed-form expression for transmittance-weighted ray integration. This representation is less expressive than full 3D Gaussians, and its BVH-based implementation still limits runtime efficiency. 2DGS [Huang et al., 2024a] replaces 3D Gaussians with 2D surfels, which enables exact ray-surface intersection tests. HTGS [Hahlbohm et al., 2025] restores full Gaussian expressions but retains the ray-splat association mechanism from 2DGS. Because the association is still defined in screen space, projective exactness is not guaranteed for wider FoVs. Our contribution is to derive closed-form bounds and an end-to-end rendering pipeline in a camera-agnostic angular domain (see details in Sec. D); Sec. D.3 provides a detailed computational comparison with the parameterizations and rendering procedures used by 2DGS and HTGS.

3DGUT [Wu et al., 2025], developed after 3DGRT, improves runtime by replacing BVH traversal with splatting-based particle association and thereby reaches frame rates comparable to 3DGS. Its acceleration, however, depends on the Unscented Transform (UT), a sampling-based approximation for estimating 2D conics in projective image space. GOF [Yu et al., 2024b] has an analogous limitation because it applies the EWA construction directly to estimate 2D bounding conics and projected centers for association. Under extreme FoV, inaccurate AABB estimation can induce incorrect tile-to-AABB associations and visible grid-line artifacts (see Fig. 1 Left).

In contrast, our method performs ray-particle association directly between 3D frustums (see Fig. 1 Right). This formulation preserves the geometric exactness of ray-based rendering for arbitrary camera models while maintaining rendering throughput comparable to 3DGS. Tab. 1 summarizes representative methods that address projective approximation error and specifies their differences along rendering, association, primitive, FoV, efficiency, and exactness dimensions.

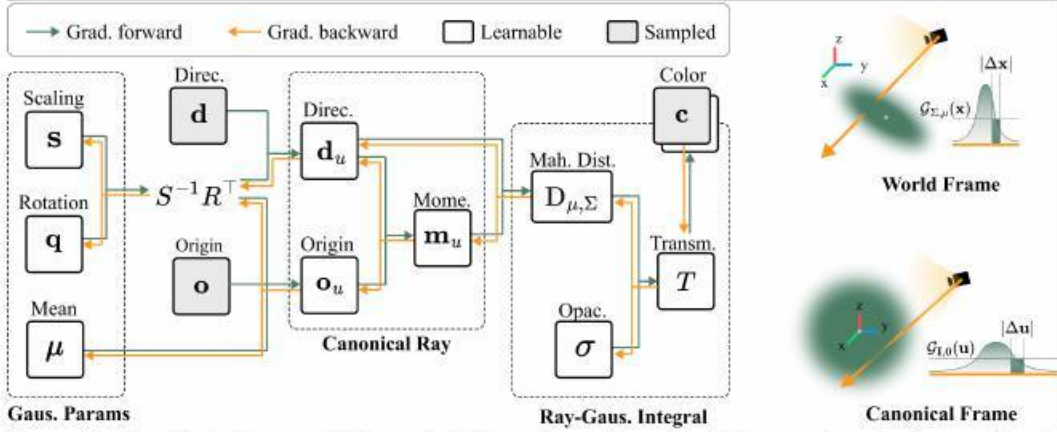


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Forward evaluation and backward gradient propagation from the Gaussian parameters s , q , μ , and opacity coefficient σ to the transmittance. (Right) Ray integral of 3D Gaussian density, where the green box denotes the product between density and ray-segment length (see Eq. B.5).

3 METHOD

We propose 3DGEER, a Gaussian rendering framework that jointly targets geometric exactness and real-time execution for arbitrary-FoV cameras. The framework addresses two principal limitations of prior methods: *projection approximation* in splatting-based rendering and *BVH overhead* in ray-based rendering. It consists of three coupled components:

- Ray-based rendering formulation (Sec. 3.1): a differentiable closed-form integral of Gaussian density along 3D rays, which removes the linearization error introduced by projection-based splatting.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a ray-particle association scheme that replaces BVH traversal with exact frustum-particle tests, providing both geometric accuracy and computational efficiency.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that parameterizes arbitrary FoV camera models, accelerates PBF computation, and improves reconstruction quality.

The Appendix supplies the formal details: (i) preliminaries and notation (Sec. B.1), (ii) the proof for Sec. 3.1 that connects maximum response with projective exactness (Sec. B), (iii) complete backward-gradient derivations and numerical-stability analysis (Sec. C), and (iv) the derivation of geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

A central requirement for geometrically exact Gaussian rendering is a closed-form expression for the density and color accumulated along a ray. We satisfy this requirement by transforming each anisotropic world-space Gaussian into a canonical coordinate system in which the density is isotropic. Under this transformation, the transmittance integral becomes a measure-preserving change of variables and admits a compact closed-form expression.

Canonical transformation. For each Gaussian with mean μ , rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which maps the canonical coordinate $\mathbf{u}$ to the world coordinate $\mathbf{x}$. The differential relation $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ incorporates the Jacobian determinant into the density measure (see Fig. 2 Right). Thus, every Gaussian is expressed by the common isotropic density

$$\mathcal{G}_{I,0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, its canonical form is $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \boldsymbol{\mu}), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The corresponding accumulated transmittance is

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu, \Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu, \Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular Mahalanobis distance between the ray and the Gaussian center.

Discussion. This expression shows that transmittance is a function only of the ray–Gaussian distance in canonical space, which yields an efficient closed-form evaluation that is exact under the stated projective geometry. The same functional form coincides with the “maximum response” heuristic used in prior work (Keselman & Hebert, 2022; Moënné-Loccoz et al., 2024); our derivation specifies the geometric conditions under which that heuristic is projectively exact (see Sec. B). In addition, the closed-form derivation and its backward-gradient chain (see Fig. 2 Left and Sec. C) avoid the numerical instabilities observed in prior ray-based work (Yu et al., 2024b) and remove the need for post-processing steps such as filtering degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For ray-based Gaussian rendering, both reconstruction accuracy, especially under large-FoV camera models, and runtime efficiency are determined by the ray–particle association procedure. This component is comparatively underspecified in prior work. We formulate association as a mapping from Camera Sub-Frustums (CSFs) to Particle Bounding Frustums (PBFs), where each PBF is the tight frustum enclosing one Gaussian at a prescribed distance from its center (Fig. 3). A Gaussian is assigned to a CSF exactly when its PBF intersects that CSF. The resulting CSF-associated Gaussian set is then used as the contributor set for all rays contained in the CSF.

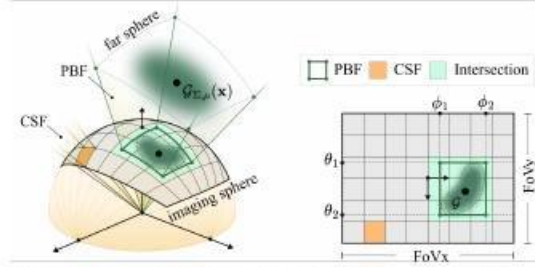


Figure 3: **PBF-CSF Association.** (Left) PBF specified by four planes tangent to a 3D Gaussian ellipsoid. (Right) Intersection between the PBF and CSF after unfolding onto the BEAP imaging plane.

To compute a PBF efficiently, we parameterize each camera-space direction by two spherical angles (see Fig. E.1):

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ is the normalized direction vector associated with the camera-space point $\mathbf{d}_c$. The variables θ and ϕ are the horizontal (xz) and vertical (yz) incidence angles of the ray, respectively, and each lies in $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. A PBF is specified by four planes: two planes parameterized by θ and two planes parameterized by ϕ (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ define the PBF angular interval.

Given a camera extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world transformation matrix H (see Eq. B.4), we transform the view-space angular planes into canonical space with the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$. This representation converts the PBF construction into constraints on canonical planes. For the θ bounds, the canonical plane is

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2}(VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, written as a column vector.

For a Gaussian ellipsoid represented by a λ -standard-deviation contour, the local-frustum angular bounds are obtained by solving quadratic tangency constraints between the transformed planes and the ellipsoidal contour:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ denotes the Hadamard (element-wise) product, and λ controls the contour spread and is typically set to 3. The same tangency constraint is applied to $\mathbf{g}_v^\top$ to obtain the corresponding $\phi_{1,2}$ bounds. The roots of the resulting quadratic equations give the tangent values for $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$, which define the PBF angular limits. Solving these quadratic equations in canonical space preserves the true camera-space 3D covariance Σ_c and mean μ_c of the Gaussian. For the θ bounds, $c = \tan \theta_{1,2}$ satisfies

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This quadratic system can be solved efficiently (see Sec. D.1). It removes the intermediate conic approximations used by EWA and UT, and it also avoids the screen-space constraints in HTGS and 2DGS that restrict their FoV.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

We introduce BEAP as the image representation used by the association and supervision pipeline. First, BEAP represents large-FoV images without discarding FoV. Second, each image tile and its corresponding CSF share the same parameterization by two θ bounds and two ϕ bounds, which enables efficient PBF association and regular GPU parallelism. Third, the induced ray sampling is more uniform in 3D than conventional projective rendering, which empirically improves training efficiency and reconstruction quality.

Specifically, rays are sampled uniformly in the spherical angular coordinates (θ, ϕ) defined in Eq. 6. They are then mapped to a discrete $w \times h$ image grid by a linear transformation with scale and center-shift terms:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ denote the horizontal and vertical camera field of view. As shown in Fig. 4, uniform sampling in (θ, ϕ) produces balanced spatial coverage within the view frustum. This differs from projective sampling, which allocates excessive samples to peripheral regions, and from equidistant projection, which oversamples the image center.

4 EXPERIMENTS

This section evaluates 3DGEER on datasets containing both pinhole and fisheye camera models. Unless otherwise specified, experiments use an RTX 4090 GPU with 64GB of RAM. Sec. G provides the complete implementation and evaluation configuration.

Datasets. For generic distorted cameras, where projective approximation error is most consequential, we evaluate all scenes from Fisheye ZipNeRF (Barron et al., 2023), five scenes from ScanNet++ (Yeshwanth et al., 2023), and two scenes from Aria (Lv et al., 2025). The ZipNeRF and ScanNet++ cameras provide 180° diagonal FoV, whereas Aria provides a 110° circular FoV. For pinhole evaluation, we use Pinhole ZipNeRF and MipNeRF360 (Barron et al., 2021).

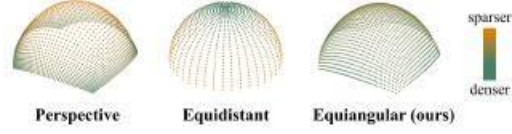


Figure 4: **Comparison of Ray Distributions under Varying Projections.** Pixels from three imaging spaces are mapped to the unit sphere to compare their induced ray distributions. BEAP yields the most uniform angular coverage among the tested parameterizations.

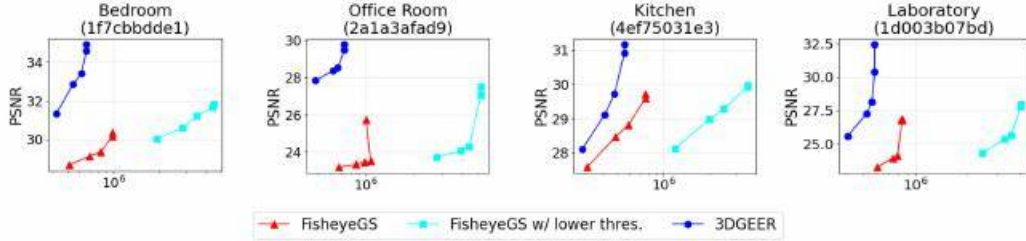


Figure 5: **PSNR $\uparrow$ Trends When Scaling-Up Gaussian Counts.** Increasing the number of splatted Gaussians does not close the projective-exactness gap; full metrics are reported in Fig. K.3 and Tab. K.3



Figure 6: **Extreme-FoV Generalization.** 3DGEER reconstructs complete scenes using fewer Gaussians, whereas FisheyeGS exhibits severe degradation under the wider-FoV evaluation.

Evaluation Metrics and Protocol. We report PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, runtime performance (FPS $\uparrow$), and Gaussian counts $\downarrow$. To avoid bias from different internal image parameterizations, every rendered result is projected back to the original image space for full-FoV evaluation after 30k training iterations.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

Celarek et al. [2025] argues that increasing the Gaussian count, up to 1M primitives, can reduce the linear-approximation gap between splatting and ray-based methods. That observation has been demonstrated only on small object-centric scenes and does not characterize wide-FoV distortion. We therefore scale the splatting-based fisheye baseline FisheyeGS on ScanNet++ by lowering its densification threshold, which produces 3.6–5M Gaussians on an H200 GPU.

Fig. 5 shows that, at matched Gaussian counts, FisheyeGS (red) remains separated from 3DGEER (blue) by a clear PSNR gap from 4k to 30k iterations, after which densification usually saturates. The scaled FisheyeGS variant (light blue) requires substantially more memory and computation, yet saturates at 29.3 PSNR, whereas 3DGEER reaches 32.1 PSNR with only 550–700K Gaussians. These results indicate that the residual error is associated with projective approximation rather than merely insufficient primitive count. Full statistics are reported in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We evaluate 3DGEER against representative baselines on fisheye and pinhole benchmarks. The protocol includes dataset-specific analyses so that distortion, camera transfer, and standard narrow-FoV performance are each measured explicitly.

ScanNet++. Tab. 2 shows that 3DGEER outperforms all baselines on the reported metrics. To quantify spatially varying distortion, we evaluate central and peripheral regions separately (protocol in Sec. G). Baselines retain reasonable accuracy near the center but lose fidelity in the periphery; 3DGEER maintains higher quality across both regions and produces sharper object boundaries with

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained either on the full FoV or on the central crop, then evaluated on full, central, and peripheral regions to measure robustness to spatially varying distortion. Region-wise metrics are weighted by pixel masks; LPIPS is omitted from regional evaluation because it is not directly decomposable by region. Tab. K.2 reports the full statistics.

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Table 3: **Comparison on the ZipNeRF Dataset.** The protocol evaluates cross-camera generalization: each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and is then tested separately on Fisheye and Pinhole views. Tab. K.9 reports the full statistics.

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
Test Data Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

fewer artifacts (Fig. K.4). Even when trained only on central crops, 3DGEER exceeds FisheyeGS across all evaluation regions.

For qualitative extreme-FoV generalization (Fig. 6), methods are trained on the original dataset FoV (180° diagonal FoV) and evaluated at a wider circular FoV (180° FoV). 3DGEER reconstructs complete scenes with fewer Gaussians, whereas splatting-based methods such as FisheyeGS degrade markedly in peripheral regions. Additional extreme-FoV visualizations are provided in Fig. K.1 K.2

ZipNeRF. The dataset provides frame-wise aligned pinhole and fisheye views. Beyond standard fisheye evaluation, we conduct *cross-camera generalization*—training on fisheye and testing on pinhole, and vice versa—to assess robustness. As shown in Tab. 3 3DGEER consistently outperforms all methods across metrics, fully exploiting large-FoV training data via geometrically exact rendering. Trained on pinhole, 3DGEER still yields notable PSNR gains on both views. In contrast, FisheyeGS performs well only on pinhole-to-pinhole, and 3DGUT drops sharply when tested on fisheye after pinhole training. Overall, EVER and 3DGEER generalize better due to geometric exactness, while 3DGEER is superior in all cases. Qualitatively (Fig. 7), 3DGEER recovers finer details and shows stronger resistance to artifacts in the periphery in the most challenging pinhole-to-fisheye setting.

We also evaluate 3DGEER on **MipNeRF360** under standard narrow-FoV conditions. Tab. 4 reports that 3DGEER achieves the best results across all metrics. It runs more than $5\times$ faster than exact ray-marching or ray-tracing methods such as EVER and 3DGRT, and it is the only projective-exact method whose runtime is comparable to 3DGS. It also improves PSNR, SSIM, and LPIPS relative to projective-approximation baselines such as 3DGS and 3DGUT. Additional quantitative results and

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Quality metrics are reported together with FPS measured on an RTX 4090 by default. Values taken from original papers are marked with $\dagger$ (RTX 6000 Ada) and $\ddagger$ (RTX 5090).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	52 $\dagger$ / 68 $\ddagger$
3DGUT	27.26	0.810	0.218	265 $\dagger$ / 317 $\ddagger$
3DGEER (Ours)	27.76	0.821	0.210	327

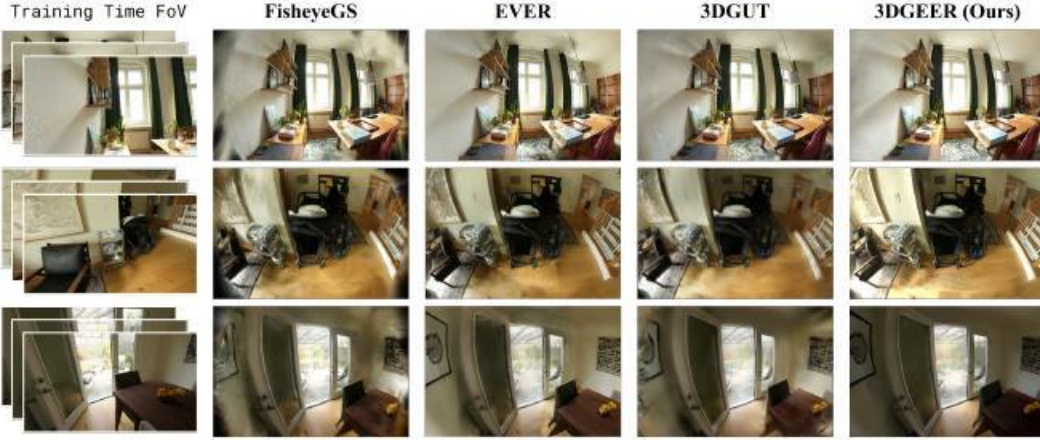


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** Performance differences are pronounced in the most challenging Pinhole (PH) train – Fisheye (FE) test setup, where our method demonstrates a significant advantage.

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** Using splatting-based transmittance increases the Gaussian count due to larger approximation error, yet the perceptual gap remains compared with our projective-exact formulation (Tab. K.7 shows the full stats).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

visualizations for **Aria**, an egocentric fisheye dataset with moderate 110° circular FoV and low-light imagery, are provided in Sec. J.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We analyze runtime on RTX 4090 by comparing 3DGEER with splatting-based methods, using 3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**. The ray-based rendering stage is slightly more expensive than splatting, but PBF association substantially reduces Gaussian duplication and sorting overhead (detailed timings in Tab. K.4). Consequently, the full pipeline achieves efficiency comparable to optimized splatting implementations.

PBF produces tight bounds: each tile is associated with only ~ 475 Gaussians on average, which is $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6). This reduced association set accounts for the $2.5\text{--}5\times$ speedup in the association stage (Tab. K.5). Projection-based schemes such as EWA and UT instead conservatively overestimate screen-space extent, increasing overdraw and degrading memory access locality. Even with SnugBox acceleration (Hanson et al. 2025), these schemes remain affected by miscentered density approximations and can intensify artifacts (Fig. 1). In addition to improving speed, PBF preserves geometric accuracy and numerical stability by removing projective approximation from association. Tab. K.6 further shows that PBF remains artifact-free even when ray-based rendering Gaussian fields trained with alternative association schemes.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of the projective-exact transmittance formulation in Eq. 4 we replace it with the standard local-affine splatting approximation, namely the 2D conic approximation used by 3DGS and FisheyeGS.

Rendering-Time Approx. To expose the difference between the two rendering formulations, we replace transmittance with the splatting approximation only at test time while retaining all projective-

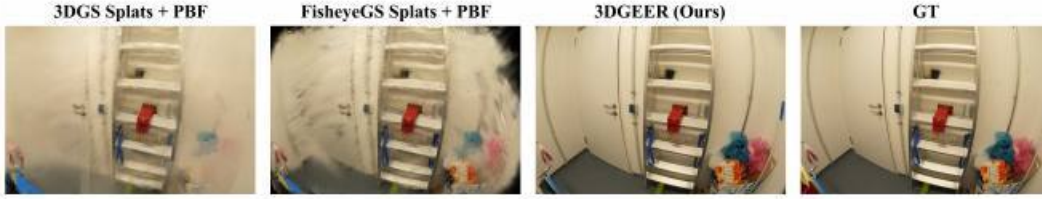


Figure 8: **Rendering-Time Transmittance Function Replacement.** Replacing projective-exact transmittance with a splatting approximation only at test time produces inconsistent support estimates and mismatched culling.

Table 6: **BEAP vs. Alternative Supervision Schemes.** BEAP outperforms the alternative projection-based supervision schemes on all reported ScanNet++ metrics. Tab. K.8 reports the full statistics.

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

exact components during training. Fig. 8 shows that splat-approximated transmittance combined with exact projective association creates inconsistent support estimates and therefore mismatched culling. This controlled mismatch explains the gridline artifact observed in 3DGUT (Fig. 1), where the inconsistency occurs in the reverse direction: ray-based transmittance is paired with conic-approximated association.

Training-Time Approx. Tab. 5 reports the ScanNet++ training-time replacement experiment. When splatting-based transmittance is used during training while association remains projective-exact, optimization and densification partially compensate for the approximation error and the cross-view inconsistency introduced by splatting. The higher loss produces stronger gradients and increases the final Gaussian count by 1.6–2.4 $\times$. Nevertheless, the perceptual gap remains despite the additional primitives, indicating that densification alone does not fully correct the approximation error.

4.5 BEAP ON/OFF STUDY

We compare BEAP supervision with other common projection models on ScanNet++. The projective-exact integral and PBF association are fixed, and only the supervision ray-sampling distribution is varied among pinhole, equidistant, and BEAP. Tab. 6 and Fig. K.5 show that BEAP-based supervision achieves the best value for every metric. Its more uniform angular sampling yields smoother and more consistent gradients, which improves convergence on fine-scale structure.

The effect is particularly pronounced for pinhole parameterizations. At fixed resolution, the distribution of information across sampled rays can be more important than increasing the nominal FoV. In perspective-domain training, full-FoV supervision can even underperform central-crop supervision (Tab. 6 rows 1–2), because large-FoV perspective images (Fig. K.2 Left) have highly non-uniform angular sampling: informative content is concentrated near the center, whereas the less informative periphery occupies most pixels and is disproportionately oversampled.

5 CONCLUSIONS

We presented 3DGEER, a closed-form framework for projective-exact volumetric Gaussian rendering derived from first principles. The rendering formulation is paired with an efficient PBF construction for ray-particle association, which enables high-throughput execution while preserving projective-geometric exactness. We further introduced BEAP as an angular image representation for applying supervision under wide-FoV camera models. Experiments across multiple datasets show

540 that 3DGEER consistently improves over state-of-the-art methods, with the largest gains occurring
541 in wide-FoV regimes where projection approximations are most severe.
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593

ETHICS STATEMENT

This work uses publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and does not introduce new human-subject data, private data, or personally identifiable information. The method is intended to advance 3D scene reconstruction and rendering, with potential applications in XR, robotics, and autonomous driving. We do not identify direct harmful consequences from the experiments reported here, but high-fidelity 3D reconstruction systems can raise privacy concerns if applied to sensitive environments. Accordingly, our evaluation is restricted to research datasets that are widely used in the vision community.

REPRODUCIBILITY STATEMENT

We provide the information needed to reproduce the technical and empirical components of the paper. The mathematical derivations and proofs are included in the Appendix (Secs. B, C and D). The experimental datasets are publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF). Preprocessing, training protocols, and evaluation metrics are specified in Sec. G. Upon publication, we will release the implementation together with training and evaluation scripts.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	2
2.1 Splatting-Based Gaussian Rendering	2
2.2 Ray-Based Gaussian Rendering	3
3 Method	4
3.1 Rendering with Exact Projective Geometry	4
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	6
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	7
4.3 Comprehensive Runtime Analysis	9
4.4 Local Affine Approximation vs. Projective-Exact Integral	9
4.5 BEAP On/Off Study	10
5 Conclusions	10
a Ethics Statement	12
b Reproducibility Statement	12
c LLM Usage Statement	18
A Appendix: Exactness and Limitation	18
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	18
B.1 Preliminaries And Notations	18
B.2 Differentiable Rendering with Projective Exactness	19
B.3 Ray Integral of Isotropic 3D Gaussian	20
C Appendix: Cookbook of Gradient Computation	21
C.1 Chain Rule	21
C.2 Two-Stage Propagation	21
C.3 Gradients for Canonical Ray	21
C.4 Gradients for Whitening Matrix	22
C.5 Gradients for Scaling and Rotation	22
C.6 Gradients for 3D Mean	22
C.7 Numerical Stability of Our Differentiable Framework	22

864	D Appendix: Additional Details in Association	23
865	D.1 Particle Bounding Frustum (PBF) Solver	23
866	D.2 Details in Computing CSF-PBF Intersection	24
867	D.3 Relation to Prior Works	25
868		
869	E Appendix: BEAP Image Inverse Interpolation	25
870	Pinhole Camera Model	26
871	KB Fisheye Model	26
872		
873	F Appendix: High-level Algorithmic Summary	27
874		
875	G Appendix: Additional Implementation Details	27
876	Baselines	27
877	Implementation Details	27
878	Central and peripheral regions in ScanNet++ evaluation	28
879	Cross-camera generalization evaluation on ZipNeRF	28
880		
881	H Appendix: EWA Splatting Preliminaries	28
882	From camera coordinates to ray coordinates	28
883	Local affine approximation	28
884	Splatting approximation error	29
885		
886	I Appendix: Advantage over Prior Ray-Tracing Methods	29
887		
888	J Appendix: Results on Egocentric Dataset	30
889		
890	K Appendix: Additional Material and Results	31
891	K.1 Additional Material.	31
892	K.2 Additional Results.	31
893		
894		
895		
896		
897		
898		
899		
900		
901		
902		
903		
904		
905		
906		
907		
908		
909		
910		
911		
912		
913		
914		
915		
916		
917		

LLM USAGE STATEMENT

A large language model (ChatGPT, OpenAI) was used as a writing assistant. Its use was limited to (i) improving manuscript clarity and readability, (ii) suggesting stylistic revisions for figure captions and section transitions, and (iii) proposing alternative phrasings for technical descriptions without changing the scientific content.

The LLM did not contribute to research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and empirical results are the work of the authors, who take full responsibility for the paper.

A APPENDIX: EXACTNESS AND LIMITATION

We use the term exactness to mean *eliminating linear approximation in projective geometry* during both the “rendering” and “ray-particle association” procedures. Under our setting, the corresponding errors do not vanish simply by increasing the number of Gaussians (see Sec. 4.1). We do not claim exactness in the full physical volumetric sense. The framework still uses approximations for ordering, self-attenuation, and overlap handling, which constitute its main limitations.

Limitations. Our formulation integrates each Gaussian independently. It therefore does not explicitly solve self-occlusion, and visibility ordering is approximated by depth-based sorting. As shown in the supplementary videos, this approximation can produce occasional popping artifacts. To our knowledge, EVER (Mai et al. 2025) is the only method that analytically resolves ordering and self-occlusion by approximating each Gaussian as a constant-density ellipsoid, but this design substantially reduces rendering speed.

Prior ray-tracing-based Gaussian renderers, including Fuzzy Metaballs (Keselman & Hebert, 2022) and subsequent methods such as 3DGR/UT (Moënné-Loccoz et al. 2024; Wu et al. 2025), use the “maximum-response” assumption for ray-Gaussian interaction. This assumption omits ordering and self-attenuation, yet has been shown to introduce negligible error in practice (see Celarek et al. (2025) Fig. 9-11 / Sec. 7.1) regardless of the number of Gaussians. Thus, the depth-based ordering used in our method can achieve the same practical accuracy as existing ray-tracing-based Gaussian renderers.

Future work. Although ordering and self-occlusion have limited practical impact in current Gaussian-based ray tracing, they remain the primary obstacles to full volumetric correctness. A natural extension is to incorporate analytic or learnable self-attenuation models, potentially drawing on ellipsoidal integration as in EVER (Mai et al. 2025), while avoiding its high runtime cost. A second extension is a more principled ordering model, which could reduce rare popping artifacts under extreme configurations. These limitations are orthogonal to the projective-exact rendering and association contributions, and resolving them would improve robustness in large-scale or highly dynamic scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We adopt the standard parameterization of 3D Gaussian primitives introduced in prior work (Zwicker et al. 2001). Each 3D Gaussian distribution is specified in world coordinates by a mean position $\mu \in \mathbb{R}^3$ and a covariance matrix Σ :

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where the normalization constant ρ is set to $\sqrt{2\pi}$, and $\mathbf{x} \in \mathbb{R}^3$ denotes a world-space point. This density differs from the forms used in recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which omit the determinant term $|\Sigma|^{1/2}$ to simplify splatting or association. The covariance is parameterized as $\Sigma = RSS^\top R^\top$, given a scaling matrix $S = \text{diag}(s)$

constructed from the scaling vector $\mathbf{s} \in \mathbb{R}^3$, and a rotation matrix $R \in \text{SO}(3)$ derived from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

For a single 3D Gaussian particle and a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, the contributed transmittance is defined as

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient associated with the Gaussian.

For a depth-sorted set of 3D Gaussian particles contributing to a ray, the rendered color C is computed by alpha compositing:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ gives the view-dependent color of the i -th Gaussian from optical center $\mathbf{o}$, parameterized with spherical harmonics (SH) (Kerbl et al., 2023). Minimizing the reconstruction loss between rendered and observed ray colors jointly optimizes $\mathbf{s}, \mathbf{q}, \mu$, the opacity coefficient σ , and the SH coefficients through backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The central technical problem in projective-exact Gaussian rendering is to obtain a closed-form ray integral for Gaussian density and color. We solve this problem by transforming each ray, relative to each Gaussian, into a canonical coordinate system in which the transmittance integral in Eq. B.2 is preserved and the Gaussian density becomes isotropic. The resulting ray color is then derived directly from this canonical representation.

We define a canonical coordinate system shared by all 3D Gaussian primitives. For a given Gaussian, a canonical-space point $\mathbf{u} \in \mathbb{R}^3$ is mapped to world coordinates by

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

Under this mapping, an infinitesimal ray segment $\Delta \mathbf{u}$ in canonical coordinates corresponds to the world-space segment $\Delta \mathbf{x} = RS \Delta \mathbf{u}$.

To preserve the transmittance contributed by each Gaussian across coordinate systems, the elementary transmittance, defined as density multiplied by segment length, must remain invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint absorbs the Jacobian determinant into the integration measure. Since $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$, the corresponding canonical density is

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.6})$$

which is isotropic and shared across all primitives in canonical space.

Therefore, the exact closed-form transmittance of each Gaussian along $\mathbf{r}(t)$ can be computed by integrating the canonical isotropic density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$. Contributions with $t < 0$ are negligible unless the optical center lies within the 3-sigma region; in such cases, existing 3D Gaussian rendering frameworks cull the Gaussian during optimization.

The integral admits a *closed-form* expression parameterized by the perpendicular distance from the ray to the Gaussian center, as stated in Theorem 1

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ is the perpendicular distance from the Gaussian center, which is the canonical-space origin, to the transformed ray $\mathbf{r}_u(t)$. Equivalently, this term is the minimum *Mahalanobis*

distance from any point on the world-space ray to the Gaussian, and therefore measures the closest approach of the ray to the particle distribution.

It remains to compute the perpendicular distance from the canonical origin to the transformed ray. We express the transformed ray by its direction vector $\mathbf{d}_u$ and moment vector $\mathbf{m}_u$:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1}R^\top) (\mathbf{o} - \boldsymbol{\mu})$ denotes the optical center in canonical space. Based on this, the square of $D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d})$ can be efficiently calculated as:

$$(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

By substituting this distance square into Eq. [B.7](#) and then the derived transmittance value into Eq. [B.3](#) we compute the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *For a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form value proportional to*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where D denotes the perpendicular distance from the Gaussian center to the ray.

Proof. We first translate the coordinate system so that the Gaussian $\mathcal{G}$ is centered at the origin, and we apply the same translation to the ray $\mathbf{r}_u$. We then rotate the coordinate system so that the ray direction aligns with the z -axis. Because the integral is translation invariant and the standard Gaussian is isotropic, these transformations leave the integral value unchanged.

The translated standard isotropic 3D Gaussian is

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. [B.6](#). Let the z -axis-aligned ray be

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ \mathbf{o}_v \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

The ray-Gaussian integral is then

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Therefore, the integral is proportional to $\exp(-\frac{1}{2}D^2)$, as stated in the theorem. $\square$

The canonical-space density shared by the 3D primitives (see Eq. [B.6](#)) has the same form as the isotropic case. When the world-coordinate ray $(\mathbf{o}, \mathbf{d})$ is provided to D , the value also depends on the world-space Gaussian parameters $\boldsymbol{\mu}, \Sigma$, which define the transformation into canonical coordinates. Thus, for each Gaussian contributing to the ray, the transmittance is

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. [B.7](#) when $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ is the ray-Gaussian transmittance (Eq. B.7), where σ is the opacity coefficient, and α is the ray-Gaussian integral. We denote $\kappa = D_{\mu, \Sigma}^2$ as the minimal squared Mahalanobis distance from the ray to the Gaussian distribution, S as the Gaussian scaling, and R as the rotation matrix in the world space. Additionally, given $\Sigma = RSS^\top R^\top$ as the 3D covariance matrix of the Gaussian in the world space, we have W_{pca} as the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad \text{where } S^{-1} = \text{diag}\left(\frac{1}{s}\right). \quad (\text{C.1})$$

As illustrated in Fig. 2 we can apply the chain rule to find derivatives *w.r.t.* the Gaussian parameters, e.g, scaling vector s , rotation quaternion q , and mean μ :

$$\frac{\partial T}{\partial s} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial s}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial q} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial q}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

To reduce computational overhead in shared propagation, we design our gradient computation following the structured two-stage approach of 3DGS (Kerbl et al. 2023), comprising `RenderCUDA` (RC) and `PreprocessCUDA` (PC). The former performs ray-wise accumulation, efficiently accelerated by our Particle Bounding Frustum (PBF) association (see Section 3.2), while the latter operates with Gaussian-wise complexity, leveraging view frustum culling for additional speedup.

According to Eq. B.7 we can respectively write the gradient *w.r.t.* opacity coefficient and the shared partial derivative for Gaussian parameters in RC as:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients to the Gaussian parameters can then be back-propagated from the squared distance, e.g., $\frac{\partial \kappa}{\partial s}, \frac{\partial \kappa}{\partial q}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We then backpropagate gradients to the canonical ray parameters: the origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$, as defined in Eq. B.8 in our paper. These quantities directly influence the Mahalanobis distance in Eq. B.9 and are critical for capturing canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = -\left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ denotes the world space optical center, and $\mathbf{d}$ denotes the ray direction. Once the gradients *w.r.t.* the ray parameters are obtained, we further propagate them to the PCA whitening matrix W_{pca} via outer products with the world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \frac{\partial \kappa}{\partial W_{\text{pca}}} &= \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} + \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here, the gradient backpropagated through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, while the gradient through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`. These two components are then summed to yield the complete gradient *w.r.t.* the PCA transformation matrix W_{pca} .

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\left. \frac{\partial W_{\text{pca}}^{i,j}}{\partial s_k} \right|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

To derive gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we can write the whitening matrix as:

$$W_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1} \left(\frac{1}{2} - q_j^2 - q_k^2 \right) & \frac{1}{s_1} (q_i q_j + q_r q_k) & \frac{1}{s_1} (q_i q_k - q_r q_j) \\ \frac{1}{s_2} (q_i q_j - q_r q_k) & \frac{1}{s_2} \left(\frac{1}{2} - q_i^2 - q_k^2 \right) & \frac{1}{s_2} (q_j q_k + q_r q_i) \\ \frac{1}{s_3} (q_i q_k + q_r q_j) & \frac{1}{s_3} (q_j q_k - q_r q_i) & \frac{1}{s_3} \left(\frac{1}{2} - q_i^2 - q_j^2 \right) \end{pmatrix}. \quad (\text{C.11})$$

As a result, we derive the following end gradients for the quaternion $\mathbf{q}$ as:

$$\begin{aligned} \left. \frac{\partial W_{\text{pca}}}{\partial q_r} \right|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_i} \right|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_2} & \frac{q_k}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \\ -\frac{2q_j}{s_3} & \frac{q_r}{s_3} & \frac{q_i}{s_3} \end{pmatrix}, \\ \left. \frac{\partial W_{\text{pca}}}{\partial q_j} \right|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_i}{s_3} \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_k} \right|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_j}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

C.6 GRADIENTS FOR 3D MEAN

Meanwhile, we compute end gradients *w.r.t.* the Gaussian 3D mean $\boldsymbol{\mu}$ as:

$$\left. \frac{\partial \kappa}{\partial \boldsymbol{\mu}} \right|_{\text{PC}} = -W_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choice of forward formulation, backward gradient computation, and intermediate cache variables critically affects numerical stability. Prior approaches such as GOF (Yu et al., 2024b) suffer from instability in backward propagation and therefore rely on additional 3D filtering heuristics to stabilize training. In contrast, our closed-form formulation, together with its exact gradient chain, avoids such instability altogether, eliminating the need for any auxiliary filtering.



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) when degenerated Gaussians are not filtered in GOF, and (Right) the stable results obtained when applying our forward and backward formulations.

The key distinction from our gradient and rendering pipeline lies in that prior methods adopt an alternative expansion of $\mathbf{m}_u^\top \mathbf{m}_u$. Ideally, disregarding numerical issues, the following two forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

However, when a 3D Gaussian has one axis scaled close to zero, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become nearly parallel. In this case, $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$, while both $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ diverge. This makes the term $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ highly prone to numerical overflow and, in practice, can even yield negative values and further lead to instable backward propagation as well.

Fig. C.1 (Left) demonstrates the numerical artifacts produced by GOF. When it does not apply additional 3D filtering to remove degenerated Gaussians, severe artifacts appear. The snowflake noise arises from numerical overflow in Eq. C.14 where negative values are prematurely returned and extremely small positive values lead to amplified Gaussian intensity. Since the EWA approximation with Jacobian and conic remains numerically stable, the artifacts mainly stem from inaccurate maximum-response computation, which also induces gridline patterns.

In contrast, our CUDA rasterizer implementation computes both the cross product and its gradients directly (Eq. B.9 (forward) and Eqs. C.6-C.9 (backward)), thereby fundamentally avoiding this numerical issue. As a result, the forward pass is artifact-free, while the backward pass yields accurate gradients. Fig. C.1 (Right) illustrates the results of applying our forward and backward formulations to GOF: even without additional 3D filtering to exclude degenerated Gaussians (i.e., flat disk-like or needle Gaussians), our approach effectively eliminates the artifacts caused by numerical instability.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 gives the detailed derivation. Section D.2 specifies implementation considerations for stable PBF computation under large-FoV and omnidirectional cameras. Section D.3 formalizes the derivational relationships and states the conditions under which prior formulas coincide with ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

From Eq. 8 define $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ as the plane transformation matrix from view space to canonical space. For the unknown θ bounds with $c = \tan \theta_{1,2}$, we have

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes the i -th matrix row expressed as a 4×1 column vector, and P_{ij} denotes its j -th element. From Eq. 9 when computing the bounding box of the 1-standard-deviation contour ellipsoid, i.e., $\lambda = 1$, the tangency constraint yields the quadratic equation

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. We write this equation as

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is a 3×3 upper-triangular matrix with entries $\mathcal{T}_{ij} = (\mathbf{f} \circ \mathbf{P}_i^\top) \cdot \mathbf{P}_j$.

Analogously, the ϕ bounds are obtained from $c = \tan \phi_{1,2}$ satisfying

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Interestingly, given a camera-view configuration $[\mathbf{R}_c \mid \mathbf{t}_c]$, we find that the parameter matrix $\mathcal{T}$ can be efficiently computed from the Gaussian's view-space covariance $\Sigma_c = \mathbf{R}_c \mathbf{R} \mathbf{S} \mathbf{S}^\top \mathbf{R}^\top \mathbf{R}_c^\top$ and 3D mean $\boldsymbol{\mu}_c = \mathbf{R}_c \boldsymbol{\mu} + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\boldsymbol{\mu}_c \boldsymbol{\mu}_c^\top)^{i,j}. \quad (\text{D.5})$$

The angular bounds, expressed by their tangent-space center and extent, are therefore

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

If the discriminant of either quadratic is negative—indicating that the camera's optical center lies inside the Gaussian ellipsoid and the angular bounds are undefined—we conservatively clamp the frustum angles using the camera's maximum field of view (FoV).

This result is computed directly from the true 3D view-space covariance. In contrast, EWA and UT use intermediate conic approximations in screen space. The PBF formulation therefore provides exact association bounds while remaining computationally efficient.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We uniformly sample spherical angles within the maximum field of view (FoV). The resulting x , y , and z coordinates, which define ray direction vectors $\mathbf{d}$, are projected to the unit sphere and then transformed into pixel space for inverse color interpolation (see Sec. E). The sampled rays are subsequently grouped by their corresponding CSFs for Gaussian association.

For most limited-FoV pinhole scenes, PBF angular bounds lie within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. Because $\tan$ is monotonic on this interval, PBF-CSF intersections can be computed by directly comparing the tangent values of the corresponding angular bounds. In wide-FoV settings, however, the PBF bounds may fall outside the monotonic interval of $\tan$ even when the incidence angle of the Gaussian center remains within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. Direct tangent-value comparison can then produce incorrect PBF-CSF intersections, for example by matching angular bounds as $[\theta'_2, \theta_1]$ (see Fig. D.1).

To recover the correct intersections, we apply a mirror transformation to the tangent values of both PBF angular bounds and CSF partitions.

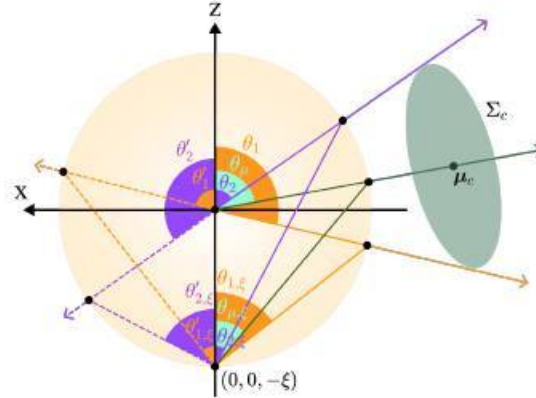


Figure D.1: Mirror Transformation of PBF Angular Bounds. Four candidate mirrored angles $\theta_{1,\xi}$, $\theta'_{1,\xi}$, $\theta_{2,\xi}$, $\theta'_{2,\xi}$ are generated from a pair of bounds. For a Gaussian centered at $\boldsymbol{\mu}_c$, the two candidates closest to its mirrored angle $\theta_{\mu,\xi}$ define the PBF bounds used for robust intersection with mirror-transformed CSF partitions.

Following the CMEI camera model (Mei & Rives 2007), we shift the optical center with a mirror parameter ξ , which gives

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ represents the mirror-transformed angle of θ on the xz -plane, using a mirror parameter ξ . We set $\xi = 1$ throughout all experiments.

For a pair of tangent values $[\tan \theta_1, \tan \theta_2]$ from the PBF solver (see Eq. D.6), the transformation generates four candidate mirror angles, as shown in Fig. D.1. The Gaussian center maps to a unique mirror-transformed angle $\theta_{\mu, \xi}$. We therefore select the two candidate angles closest to $\theta_{\mu, \xi}$, which define the effective PBF lower and upper bounds $[\theta_{1, \xi}, \theta_{2, \xi}]$. These bounds are then compared with CSF partitions to compute the correct intersections.

This transformation yields robust angular comparisons beyond the monotonic domain of the tan function. As reported in the timing analysis (Tab. K.5), the preprocessing pipeline, including mirror-angle PBF-CSF intersection and canonical transformation for all Gaussians, requires less than 0.13 ms in total. The overhead therefore remains small even for thousands of Gaussians in wide-FoV scenarios.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen-space coordinates

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation yields expressions identical to those in 2DGS (Huang et al. 2024a) and HTGS (Hahlbohm et al. 2025). This equivalence is expected because perspective screen space is a special parameterization of our angular-domain formulation.

More specifically, 2DGS and HTGS begin from a screen-space or tangent-plane parameterization and therefore assume geometric identities that hold for perspective pinhole cameras, where rays are implicitly associated with a planar image plane parallel to the z -axis. These identities do not extend to fisheye or omnidirectional cameras, for which the corresponding screen-space surface is curved. Our derivation instead starts from a camera-agnostic 3D angular parameterization and directly specifies Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). The different assumptions change the available constraints in the derivation and determine when closed-form bounds and culling guarantees remain valid for generic cameras.

The contribution is therefore not the algebraic manipulation alone, but the derivation of these bounds under weaker assumptions, without relying on screen-space structure, and their validation across both pinhole and wide-FOV camera families.

The parameterization also changes the downstream computational pipeline. Gaussian association, tiling or duplication, and sorting all depend on how Gaussians are mapped to tiles or bins. An angular CSF-based domain induces a different bucketing structure from pinhole screen-space tiling. This distinction is important for large-FoV scenes: under generic cameras, a Gaussian bound may appear partially “behind” the camera in screen coordinates (see Fig. D.1), which can make screen-space methods produce mirrored or incorrect tangent-based bounds (dashed line in Fig. D.1). We handle these cases by introducing mirror parameters in the association step (Sec. D.2) and by using the Gaussian center for sorting and correction, neither of which is required in perspective-only derivations.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To instantiate the training inputs, we sample a ray set whose angular support covers the original image FoVs and submit these rays to the CUDA renderer. For each sampled ray, we additionally evaluate the corresponding ground-truth color by interpolation, which yields geometrically aligned and spatially uniform supervision during optimization.

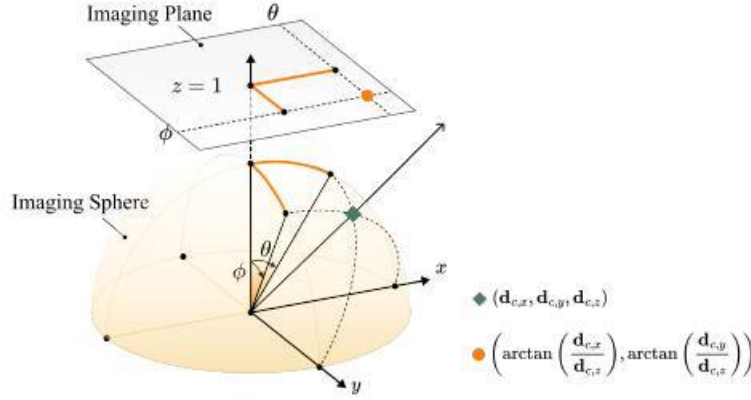


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is first normalized on the optical sphere (green spade) and subsequently represented by angular projections onto the xz and yz planes. The orange point denotes the induced (θ, ϕ) coordinate in the unfolded BEAP image domain.

The projection from sampled rays to observed images first maps the incidence-angle coordinates θ and ϕ to normalized ray-direction vectors $\mathbf{d} = (x, y, z)^\top$:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

The appropriate camera model is then applied to map the normalized ray $(x, y, z)^\top$ to the homogeneous pixel coordinate $(x_p, y_p, 1)^\top$.

Pinhole Camera Model. For a pinhole camera, the angular coordinates satisfy $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$. The resulting intrinsic projection is therefore

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. For the Kannala–Brandt fisheye model, the incidence angle φ is recovered from the projected angular components as follows:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

The model then evaluates the distorted fisheye angle according to the polynomial radial distortion parameterization

$$\varphi_d = \varphi(1 + k_1 \varphi^2 + k_2 \varphi^4 + k_3 \varphi^6 + k_4 \varphi^8). \quad (\text{E.6})$$

This distorted angle defines the pixel-space projection used during inverse color interpolation:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We provide a high-level overview of the volumetric rendering pipeline in Alg. 1 and Alg. 2. To make the comparison with 3DGS [Kerbl et al., 2023] explicit, newly introduced or replaced modules are highlighted in **green**, and substantially modified modules are highlighted in **yellow**.

ALGORITHM 1: GPU Software Asso. & Render.

```

Function 3DGEER ( $\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V$ ) :
  CullGaussian( $\mu, V$ )                                ▷ Frustum Culling
   $F \leftarrow \text{InitCSFs}(w, h, K)$                       ▷ Partition
   $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(\mathbf{s}, \mathbf{q}, V)$   ▷ Transform
   $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$         ▷ Sub-frustum Asso.
   $W_{pca} \leftarrow \text{PCA\_Whitening}(\mathbf{s}, \mathbf{q})$               ▷ Transform
   $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$ 
   $\mathcal{G}_{ID}, F_K \leftarrow \text{Indices (Gaussian) \& Keys (FrustumID+Depth)}$ 
  SortByKeys( $\mathcal{G}_{ID}, F_K$ )                                ▷ Globally Depth Sort
   $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
   $I \leftarrow 0$                                           ▷ Init Canvas (w/ Rays)
  for all CSFs  $f \leftarrow I$  do
    for all Rays  $i \leftarrow f$  do
       $r \leftarrow \text{GetFrustumRange}(R, f)$ 
       $\mathbf{m}_u, \mathbf{d}_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$       ▷ Transform
       $D_{\mu, \Sigma} \leftarrow \text{Dist}(\mathbf{m}_u, \mathbf{d}_u)$            ▷ Mah-distance
       $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$ 
      ▷ Alpha Blending
    end
  end
  return  $I$ 

```

ALGORITHM 2: 3DGEER Initialization and Training Framework

w, h : width and height of the training images
 K, V : intrinsics and extrinsic meta info.

```

 $\mu \leftarrow \text{SfM Points}$                                 ▷ Init Gaussian Attributes
 $\mathbf{s}, \mathbf{q} \leftarrow \text{InitAttributes}()$                   ▷ Position
 $c, \sigma \leftarrow \text{InitCoefficients}()$               ▷ Scaling, Rotation
 $\mathbf{r}_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$                   ▷ Color, Opacity
while not converged do
   $\mathbf{r}_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$                   ▷ View Rays / Colors
   $C \leftarrow 3DGEER(\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V)$     ▷ Asso. & Render
   $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$                           ▷ See Alg. 1
   $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$       ▷ Loss
  if IsRefinementIteration( $i$ ) then                  ▷ Backprop & Step
     $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Optimize}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma)$ 
    ▷ Kerbl et al. (2023)
  end
end

```

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We compare 3DGEER with representative volumetric particle rendering methods that target projective approximation error in 3DGS [Kerbl et al., 2023]. For large-FoV fisheye inputs, the baselines are FisheyeGS [Liao et al., 2024], EVER [Mai et al., 2025], and 3DGUT [Wu et al., 2025]. For pinhole settings, we additionally compare against 3DGS and 3DGRT [Moënné-Loccoz et al., 2024].

Implementation Details. For fair comparison, the ScanNet++ and MipNeRF360 experiments use initialization and hyperparameters aligned with the original 3DGS configuration. For ZipNeRF, all candidate methods use the training configurations from EVER and SMERF (Duckworth et al., 2024). As in most ray-based frameworks, our method does not use mean2D gradients for densification or pruning; it instead optimizes with gradients of the view-space mean3D . Unless stated otherwise, all models use consistent supervision strategies and architectural settings. For fisheye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al., 2025) and reproject all rendered outputs to the native KB-Fisheye image space (Kannala & Brandt, 2006) before evaluation. ScanNet++ and MipNeRF360 are evaluated at full resolution. For ZipNeRF, because of the high frame count and very large fisheye resolution, we use the maximum resolution (see Tab. 3) that allows all baselines to run reliably on a local desktop setup.

Central and peripheral regions in ScanNet++ evaluation. The central region contains rays whose angles satisfy

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}].$$

The peripheral region contains rays in the remaining angular ranges. Here, w, h are the image width and height, and f_x, f_y are the focal lengths defined by the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF provides two data modalities: raw fisheye images and corresponding undistorted frames obtained by distortion correction and central cropping. Although the frames are aligned per image, camera poses and point clouds were reconstructed independently with COLMAP (Schönbberger & Frahm, 2016) for each modality. Because COLMAP independently optimizes the global camera trajectories, a single rigid transformation is insufficient to align the two pose sets and can introduce alignment error. To avoid structure-from-motion (SfM) alignment artifacts in cross-camera evaluation, we always use the same test-time camera poses as those used during training. Cross-camera experiments vary only the intrinsic parameters while keeping extrinsic poses fixed, which isolates camera-model generalization from preprocessing-induced pose inconsistency.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. Following the coordinate definitions of EWA Splatting (Zwicker et al., 2001), the projective transformation for pinhole cameras maps camera-space points $\mathbf{t}$ to ray-space points $\mathbf{x}$ through $\tau : \mathbf{t} \rightarrow \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$\text{, where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are non-affine. Computing ray-space integrals therefore requires either expensive exact treatments (Westover, 1990; Mao et al., 1995) or local approximations (Zwicker et al., 2001; Kerbl et al., 2023).

Local affine approximation. EWA splatting (Zwicker et al., 2001) introduces a local affine approximation τ_k so that the k -indexed reconstruction kernel remains Gaussian in 2D screen space after linearization. The approximation is obtained from the first-order Taylor expansion of τ at camera-space point $\mathbf{t}_k$, so a nearby ray-space point $\mathbf{x}$ around $\mathbf{x}_k$ is approximated by

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian matrix of τ evaluated at $\mathbf{t}_k$. For the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in camera coordinates, the EWA local affine transformation represents the approximated ray-space density as



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** On the MipNeRF360 pinhole setting, 3DGEER improves visual quality over both 3DGS-based Gaussian Splatting and 3DPRT-based Particle Ray Tracing methods while maintaining high runtime efficiency. Colored boxes highlight the most visible differences.

a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$. For $\forall \mathbf{x}$ in a neighborhood of $\mathbf{x}_k$, we have

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating the 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields the 2D Gaussian $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$, the mean is $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 covariance matrix $\hat{\Sigma}_k$ is obtained from Σ_k by removing the third row and column.

Splatting approximation error. Under a general projective transformation from view space, the ray-space density is not guaranteed to remain Gaussian. The exact transformed density can nevertheless be written by substitution through Eq. [H.2](#)

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1}(\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)\right), \end{aligned} \quad (\text{H.8})$$

The approximation error can therefore be estimated by Monte Carlo integration of

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Here, x_2 denotes the ray-direction dimension in ray coordinates.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray tracing, such as 3DGRT, can be projective-exact in principle. In practice, however, 3DGRT accelerates BVH intersection tests with inscribed proxy icosahedra. These proxies are tight at vertices, but their face centers lie $\sim 20\%$ inward by the geometry of the icosahedron, which can reduce intersection accuracy. The resulting inaccurate accumulated gradients during density control can produce incorrect geometry, such as missing leaves in Fig. [I.1](#). In addition, intersection tests for distant, small, and cluttered Gaussians are constrained by floating-point precision and can introduce noisy renderings. For example, Fig. 15 of the 3DGRT paper shows visible speckles in tree and background regions in the native rendering block. On MipNeRF360, even 3DGUT with UT approximation outperforms 3DGRT. For validation, we render the same pretrained MipNeRF360-stump scene with the 3DGRT ray-tracing/rendering engine and with the 3DGEER pipeline. The PSNR $\uparrow$ /SSIM $\uparrow$ /LPIPS $\downarrow$ comparison is 25.79/ 0.745/ 0.264 for 3DGRT and 26.83/ 0.774/ 0.246 for ours, corresponding to +1.04/ +0.029/ -0.018. 3DGEER combines projective-exact rendering, efficient association, and BEAP color supervision, producing higher accuracy at substantially higher speed (Tab. [4](#)).



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method preserves sharper indoor floor textures and clearer distant tree structures in outdoor scenes.

Table J.1: **Quantitative Comparison on Aria.** Higher PSNR/SSIM and lower LPIPS indicate better performance.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Recent egocentric devices commonly use wide-FoV cameras, but existing reconstruction pipelines often lose quality either when using full-resolution images or when restricting training to cropped views. Low-light capture further increases the difficulty because small sensors and aperture-constrained lenses reduce light intake and amplify degradation.

We evaluate on the Aria egocentric dataset, which has a $\sim 110^\circ$ circular FoV. 3DGEER obtains an average PSNR of 33.5dB and outperforms FisheyeGS (32.8dB) on all reported metrics (Tab. J.1). Qualitatively, the reconstructions retain sharper peripheral details in the indoor scene (Fig. J.1 Left) and recover distant structures such as trees in the outdoor scene (Fig. J.1 Right). The +0.7 dB gain lies between the improvements on pinhole datasets (e.g., MipNeRF360: +0.3dB; ZipNeRF-cross: +2.0dB) and the gains on highly distorted wide-FoV datasets (e.g., ScanNet++/ZipNeRF 180 $^\circ$ diagonal FoV: +0.9–4.8dB), which is consistent with Aria’s moderate FoV and distortion level.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

The Supplementary Material includes code, video results, and an HTML project page.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6** 3DGS collapses to black because $\tan 90^\circ \rightarrow \infty$ creates a numerical singularity, whereas our method robustly renders views up to 180° and reconstructs the green chair where splatting-based methods such as FisheyeGS fail. In the zoomed regions of Fig. 6 our method also preserves structures such as the corridor and fridge, table and door, and window and cabinet.



Figure K.2: **Extreme-FoV Generalization.** (Left) Comparison between our method and 3DGS on extreme-FoV pinhole (170°) views after training on undistorted central regions. (Right) Zoomed-out version of Fig. 6 showing the complete circular 180° FoV and demonstrating robust generalization to out-of-distribution regions.

Table K.1: **MipNeRF360 Quantitative Results by Sequence.** Aggregate results are reported in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

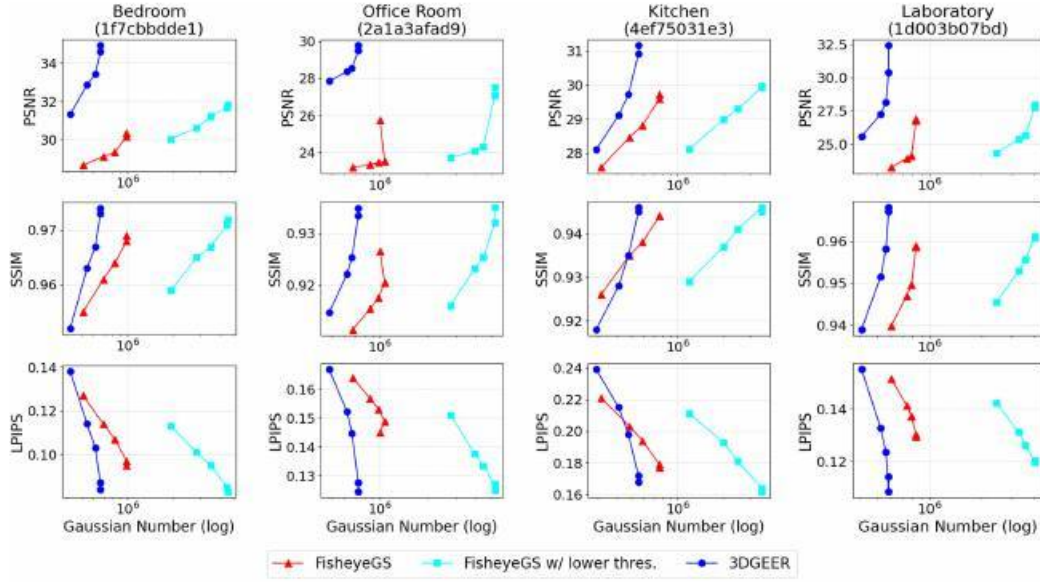


Figure K.3: **PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts.** Increasing the primitive count alone does not close the projective-exactness gap.

Table K.2: **ScanNet++ Quantitative Results by Sequence (Full FoV).** Aggregate results are reported in Tab. 2

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114

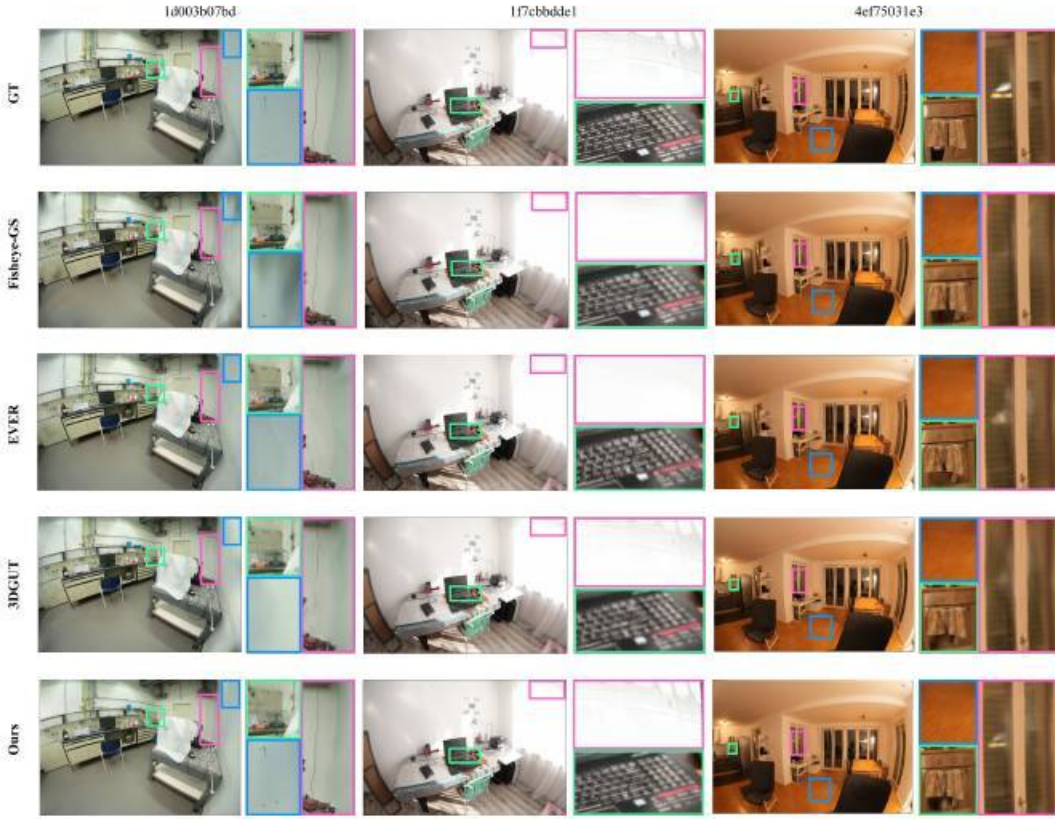


Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** Zoomed colored boxes highlight structural details and artifact-prone regions. Compared with FisheyeGS, our method reconstructs peripheral content more accurately; compared with EVER, it produces fewer artifacts and sharper details; and compared with 3DGUT, it yields crisper object boundaries and textures such as wires, keyboards, and wooden floors.

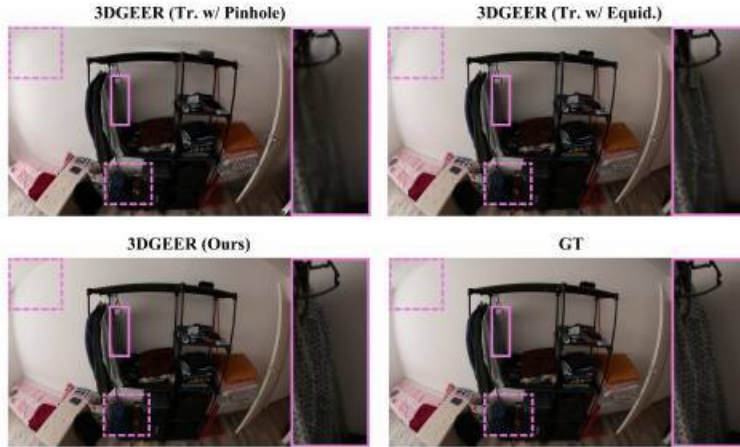


Figure K.5: **Impact of BEAP Supervision.** Visual comparisons on ScanNet++ show that BEAP's balanced angular sampling yields smoother gradients and better preservation of geometric and texture details than pinhole or equidistant sampling. Tab. K.8 reports the full quantitative statistics.

Table K.3: **Quantitative Results on the ScanNet++**. PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts. Bedroom, Office Room, Kitchen, and Laboratory in order.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Other Methods**. 3DGEER is compared with efficient splatting-based methods on MipNeRF360 and ScanNet++, including both stage-wise and total runtime.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Runtime Analysis: Association Method on ScanNet++**. PBF-based association is compared with splatting-based EWA and UT association, including variants accelerated by SnugBox within the 3DGEER pipeline.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Total	Avg. FPS ↑
		Dup.	Sort	Render			
EWA	0.13	1.17	3.27	12.69	17.84	56	
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115	
UT	0.13	0.71	1.94	7.97	11.16	90	
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228	
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251	

Table K.6: **Impact of PBF-CSF Association**. Evaluation uses the artifact-sensitive LPIPS metric. To assess cross-model robustness, each variant is also tested on models trained with the other association strategies. We additionally report rendering memory, the average number of Gaussian associations per tile, and the Δ LPIPS gap between in-method and cross-method evaluations to quantify consistency and generalization.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Training-Time Transmittance Function Replacement**. Aggregate results are reported in Tab. 5.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Impact of Supervision Space**. Aggregate results are reported in Tab. 6.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **ZipNeRF Quantitative Results by Sequence.** Alameda, Berlin, London, and NYC are reported in order. Aggregate results are shown in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220